

Informe Técnico: Arquitectura de Billeteras Deterministas y Estructuras de Derivación de Claves en Ecosistemas Blockchain

Introducción

La gestión de claves criptográficas dentro de la tecnología blockchain ha evolucionado desde el uso de billeteras no deterministas (billeteras JBOK, Just a Bunch of Keys), que requerían respaldar de forma iterativa cada dirección generada, hacia ecosistemas jerárquicos deterministas. Esta evolución estructural mitiga el riesgo de pérdida de fondos, mejora la experiencia de usuario (UX) mediante el uso de copias de seguridad de una sola vez y garantiza la interoperabilidad cross-chain. El presente informe técnico detalla los estándares fundamentales de la capa de aplicación e infraestructura criptográfica, cubriendo las especificaciones de generación mnemónica, derivación jerárquica y las rutas de derivación específicas empleadas por los ecosistemas de Bitcoin, Ethereum/EVM, Binance Smart Chain (BNB) y Solana.

1. BIP39: Código para la Generación de Claves Deterministas

El estándar BIP39 (Bitcoin Improvement Proposal 39) define el mecanismo para transformar la entropía binaria generada aleatoriamente en una secuencia legible por humanos de palabras comunes, llamada frase de recuperación o seed phrase (Bitcoin Wiki, 2025). Este proceso consta de dos fases críticas: la codificación de la entropía y la derivación de la semilla binaria mediante funciones de estiramiento de clave.

1.1. Codificación de Entropía y Suma de Comprobación

Entropía Inicial (ENT)	Checksum (CS)	Entropía Total (ENT+CS)	Palabras Mnemónicas
128 bits	4 bits	132 bits	12 palabras
160 bits	5 bits	165 bits	15 palabras
192 bits	6 bits	198 bits	18 palabras
224 bits	7 bits	231 bits	21 palabras
256 bits	8 bits	264 bits	24 palabras

2. BIP32 HDKey: Billeteras Jerárquicas Deterministas

El estándar BIP32 introduce la arquitectura jerárquica estructurada en forma de árbol, permitiendo que a partir de la semilla binaria de 512 bits generada en BIP39 se derive una secuencia infinita de claves públicas y privadas (Wuille, 2012).

2.1. Mecanismos de Derivación: Normal vs. Endurecida

La Función de Derivación de Claves Hijas (Child Key Derivation o CKD) toma una clave padre y un índice entero de 32 bits (i) para generar claves hijas deterministas. BIP32 clasifica el espacio de índices de 32 bits en dos rangos operativos:

1. Derivación Normal (Índices $0 \leq i < 2^{31}$):

Utiliza la clave pública padre y el chain code para calcular la clave hija. Permite la derivación de claves públicas hijas directamente desde una clave pública padre (x_{pub}),

2. Derivación Endurecida o Hardened (Índices $2^{31} \leq i < 2^{32}$):

Denotada frecuentemente con un apóstrofe o una "H" (ej. 0' o 0H). El algoritmo de derivación sustituye la clave pública padre por la clave privada padre dentro de la función hash de generación (HMAC-SHA512). Esto rompe el vínculo matemático entre la clave pública padre y las claves hijas, impidiendo que el compromiso de una clave hija exponga las ramas superiores de la jerarquía (Trezor KB, 2024).

3. Estructuras y Rutas de Derivación por Ecosistema

Para mitigar la fragmentación provocada por las infinitas estructuras de árboles posibles bajo BIP32, el estándar BIP44 define una estructura organizativa rígida de 5 niveles para rutas de derivación:

m / purpose' / coin_type' / account' / change / address_index

Donde 'm' denota la raíz (master key), 'purpose' se fija en '44' para billeteras multifirma/multiactivo estándar, y 'coin_type' se indexa conforme al registro oficial de parámetros de tokens SLIP-0044.

3.1. Derivación Bitcoin (BTC)

El ecosistema de Bitcoin implementa rutas específicas que no solo estructuran cuentas, sino que determinan de forma explícita el formato de los scripts y codificación de las direcciones en la capa de red:

- Legacy (BIP44): Genera direcciones tradicionales basadas en el hash de clave pública P2PKH (comienzan con 1).

Ruta: m / 44' / 0' / 0' / 0 / i

- Nested SegWit (BIP49): Encapsula el script SegWit dentro de un script de pago P2SH (comienzan con 3).

Ruta: m / 49' / 0' / 0' / 0 / i

- Native SegWit / Bech32 (BIP84): Formato SegWit nativo que optimiza las comisiones (comienzan con bc1q).

Ruta: m / 84' / 0' / 0' / 0 / i

- Taproot (BIP86): Implementa firmas Schnorr y árboles de scripts alternativos abstractos MAST (comienzan con bc1p).

Ruta: m / 86' / 0' / 0' / 0 / i

3.2. Derivación Ethereum y Cadenas EVM (ETH, BNB, Polygon)

El ecosistema de la Máquina Virtual de Ethereum (EVM), que incluye redes como Ethereum, Binance Smart Chain (BNB Chain), Polygon, Avalanche C-Chain y Arbitrum, comparte el mismo estándar criptográfico de hashing (Keccak-256) y la misma curva elíptica (SECP256k1).

- Especificación Estándar (BIP44): Registrado bajo el identificador de moneda 60'.

Ruta: m / 44' / 60' / 0' / 0 / i

A diferencia de Bitcoin, donde la billetera genera una nueva dirección de cambio (change = 1) por cada transacción para preservar la privacidad, las redes EVM se basan en un modelo de estados basado en cuentas (account-based model). Consiguientemente, el parámetro 'change' se mantiene rígidamente en 0, y se incrementa secuencialmente el parámetro 'address_index' (i) para interactuar con múltiples cuentas asociadas a la misma frase semilla (ConsenSys, 2026; MyEtherWallet, 2026). Por razones de interoperabilidad y UX, billeteras como MetaMask y

Trust Wallet aplican exactamente la misma ruta $m/44'/60'/0'/0/i$ para desplegar las claves privadas en BNB Chain y otras EVM paralelas.

3.3. Derivación Solana (SOL)

El ecosistema de Solana difiere de manera disruptiva en su capa criptográfica basal con respecto a Bitcoin y Ethereum. En lugar de la curva elíptica SECP256k1, Solana utiliza la curva elíptica Ed25519 para la firma de transacciones Edwards-curve Digital Signature Algorithm (EdDSA) (Frostbutter, 2023).

Debido a que el algoritmo Ed25519 nativo no soporta matemáticamente la derivación de claves públicas sin poseer las claves privadas intermedias (derivación normal de BIP32), las aplicaciones y librerías de Solana operan exclusivamente con derivación endurecida en todos los componentes de la ruta de acceso.

Las aplicaciones de software modernas (como Phantom o Solflare) interactúan bajo dos esquemas de ruta de derivación principales para mapear direcciones basadas en Base58 (Phantom Help Center, 2024):

- Ruta de Derivación Estándar de Solana (BIP44 Multi-Account): $m / 44' / 501' / i' / 0'$
- Ruta Alternativa Directa (Histórica / Solflare): $m / 44' / 501' / i'$

¿Por qué elegimos esta tecnología y qué alternativas descartamos?

Para justificar el desarrollo de la wallet, es vital demostrar por qué la combinación de **BIP39 + BIP32 + BIP44** es superior a otras opciones del mercado actual.

Alternativa Descartada 1: Billeteras No Deterministas (JBOK - "Just a Bunch of Keys")

¿Cómo funcionaba?: Generaba claves privadas aleatorias e independientes para cada dirección.

Por qué la descartamos: * Péssima UX y Alto Riesgo: Si el usuario realizaba 50 transacciones y generaba direcciones nuevas, tenía que hacer una copia de seguridad (backup) cada vez. Si olvidaba respaldar la última clave y perdía el dispositivo, **los fondos se perdían para siempre.**

Alternativa Descartada 2: Almacenamiento de Claves Crudas (Raw Private Keys) sin Mnemónicos

¿Cómo funcionaba?: Entregarle al usuario directamente su clave privada en formato Hexadecimal o Base58 (una cadena larga y confusa como 5KyZdBx...).

Por qué la descartamos:

Propensión a Errores Humanos: Copiar a mano 64 caracteres alfanuméricos es extremadamente propenso a errores (confundir una O con un 0, o una l con un 1). Un solo carácter incorrecto significa perder el acceso a la wallet.

La solución elegida (BIP39): Al usar un diccionario fijo de 2048 palabras, es mucho más fácil de escribir, leer y verificar por humanos. Además, el sistema de *checksum* (suma de comprobación) detecta inmediatamente si el usuario se equivocó en una palabra al intentar recuperar su wallet.

Alternativa Descartada 3: Algoritmos de Derivación Propietarios o No

Estándar

¿Cómo funcionaba?: Diseñar nuestra propia lógica matemática para ramificar las cuentas dentro de la wallet.

Por qué la descartamos:

Cero Interoperabilidad: Si nuestra wallet quiebra o la aplicación deja de funcionar, el usuario no podría mudar sus fondos a otra wallet (como MetaMask o Phantom).

La solución elegida (BIP44): Al adoptar las rutas estándar (ej. m/44'/60'/0'/0/i para EVM) , garantizamos que si el usuario mete sus 12 palabras en MetaMask, verá exactamente los mismos saldos y cuentas que ve en nuestra aplicación. Esto genera confianza y cumple con los estándares globales de la industria.

Conclusión

La integración sinérgica de los estándares BIP39, BIP32 y el modelo de rutas BIP44 conforma la infraestructura de la criptografía aplicada moderna. Mientras que las redes basadas en Bitcoin y la EVM convergen en el uso de la curva SECP256k1 con ligeras variaciones en la arquitectura de saldos frente a cuentas, redes de alta velocidad como Solana redefinen el paradigma mediante el uso estricto de la curva Ed25519 y esquemas de derivación completamente endurecidos. Comprender la matemática subyacente y la sintaxis estructural de estos estándares es imperativo para mitigar colisiones de derivación y garantizar la seguridad en el desarrollo de software blockchain.

Referencias

Bitcoin Wiki. (2024, 24 de abril). BIP 0032: Hierarchical Deterministic Wallets.

https://en.bitcoin.it/wiki/BIP_0032

Bitcoin Wiki. (2025, 27 de diciembre). Seed phrase.

https://en.bitcoin.it/wiki/Seed_phrase

ConsenSys. (2026, 28 de abril). Set m/44'/60'/0'/0 as new default derivation path.

Bump to 3.0.0. GitHub Repository. <https://github.com/ConsenSys/eth-lightwallet/pull/82>

Frostbutter, N. (2023, 12 de enero). Derive Solana Addresses. Nick Frostbutter Dev Blog. <https://nick.af/articles/derive-solana-addresses>

MyEtherWallet Help Center. (2026). HD wallets and Derivation paths.

<https://help.myetherwallet.com/en/articles/5867305-hd-wallets-and-derivation-paths>

Phantom Help Center. (2024). Supported derivation paths in Phantom wallet.

<https://help.phantom.com/hc/en-us/articles/12988493966227-Supported-derivation-paths-in-Phantom>

Tbocek, T. (2019). BIP 39 - Distributed Systems and Ledgers Lab. OST

Ostschweizer Fachhochschule. <https://dsl.i.ost.ch/lect/fs19/bip39.html>

Trezor KB. (2024). What is BIP32?: <https://trezor.io/learn/advanced/standards-proposals/what-is-bip-32-how-hierarchical-deterministic-wallets-work>

Wuille, P. (2012). BIP 0032: Hierarchical Deterministic Wallets. Bitcoin

Improvement Proposals. <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>